

Practica 7

Exercici 1 - Reproducció des de memòria interna

Codi (main.cpp)

```
#include <Arduino.h>
#include "AudioGeneratorAAC.h"
#include "AudioOutputI2S.h"
#include "AudioFileSourcePROGMEM.h"
#include "sampleaac.h"

AudioFileSourcePROGMEM *in;
AudioGeneratorAAC *aac;
AudioOutputI2S *out;

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println("Iniciant àudio...");

    // Font AAC en memòria
    in = new AudioFileSourcePROGMEM(sampleaac, sizeof(sampleaac));

    // Decoder
    aac = new AudioGeneratorAAC();

    // Salida I2S
    out = new AudioOutputI2S();

    // 🎧 Guany (Volum de 0.0 a 1.0)
    out->SetGain(0.15);

    // 🎧 PINES I2S SEGURS PER A ESP32-S3
    // Format: SetPinout(BCLK, LRCK, DOUT)
    out->SetPinout(4, 5, 6);

    // Forçar mode Mono
    out->SetOutputModeMono(true);

    // Iniciar reproducció
    if (!aac->begin(in, out)) {
        Serial.println("ERROR: No s'ha pogut iniciar el descodificador AAC");
    } else {
        Serial.println("Reproduint àudio...");
    }
}
```

```

void loop() {
  if (aac->isRunning()) {
    if (!aac->loop()) {
      aac->stop();
      Serial.println("Àudio finalitzat");
    }
  } else {
    Serial.println("Reiniciant en 2 segons...");
    delay(2000);

    // SOLUCIÓ CORRECTA: Rebobinem el punter de l'àudio a la posició
    0
    in->seek(0, SEEK_SET);

    if (!aac->begin(in, out)) {
      Serial.println("ERROR en reiniciar AAC");
    } else {
      Serial.println("Reproduint àudio de nou...");
    }
  }
}

```

Codi (.ini)

```

[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

lib_deps =
  https://github.com/earlephilhower/ESP8266Audio.git#1.9.7

```

Modificació extra en el .ini per la llibreria, per temes de compatibilitat

Descripció de la sortida del port serie

Al Monitor Sèrie, la seqüència començarà de manera immediata mostrant el missatge "Iniciant àudio...". Si la matriu de dades d'àudio és vàlida i es carrega bé en memòria, llegiràs automàticament el text "Reproduint àudio...". El programa es mantindrà en silenci comunicatiu durant tota la reproducció fins que la pista s'acabi; en aquest instant precis, el monitor registrarà les línies "Àudio finalitzat" i "Reiniciant en 2 segons...". Passat aquest breu retard, veuràs de manera cíclica el missatge Reproduint àudio de nou... seguit de la confirmació de finalització i reinici de forma infinita. En cas que hi hagués algun problema crític de descodificació, el sistema t'alertaria amb missatges d'error com "ERROR: No s'ha pogut iniciar el descodificador AAC" o "ERROR en reiniciar AAC".

Explicació del seu funcionament

Aquest programa té com a objectiu reproduir un fitxer d'àudio digital en format comprimit AAC de manera continuada, utilitzant el bus de comunicació I2S i la memòria Flash interna del propi microcontrolador. Per aconseguir-ho, es recolza en la versàtil llibreria `ESP8266Audio`, la qual proporciona les eines per gestionar tota la cadena de so digital: la classe `AudioFileSourcePROGMEM` s'encarrega de llegir el vector binari del fitxer d'àudio (`sampleaac.h`) guardat a la memòria d'execució, `AudioGeneratorAAC` actua com a descodificador del format comprimit, i `AudioOutputI2S` envia el corrent de dades resultant cap al bus I2S d'un convertidor DAC extern.

Dins de la funció `setup()`, es creen de manera dinàmica les instàncies per a aquests tres elements i es configuren els paràmetres físics de so optimitzats, en aquest cas, per a una placa de la família ESP32-S3. Primer, el codi defineix un guany (volum) moderat i segur de 0.15 per protegir l'altaveu. Seguidament, mitjançant `SetPinout()`, s'assignen els pins de control de maquinari específics de l'I2S: el rellotge de bits (BCLK) al pin 4, el selector de canal esquerre/dret (LRCK) al pin 5 i la sortida de dades en sèrie (DOUT) al pin 6. Finalment, es força el mode de sortida a "Mono" per reajustar el so i s'intenta inicialitzar el descodificador amb la instrucció `aac->begin()`.

La funció `loop()` s'encarrega d'executar el flux de so en temps real de manera no blocant gràcies a l'ordre `aac->loop()`. Aquesta funció es crida constantment mentre la reproducció està activa (`aac->isRunning()`) per anar descodificant i enviant petits paquets de dades a l'altaveu; quan la cançó arriba al seu final, `aac->loop()` retorna un fals, atura el motor i envia el senyal de finalització. En el següent cicle del programa, en veure que l'àudio s'ha aturat, s'aplica una pausa de dos segons i s'executa la comanda crucial `in->seek(0, SEEK_SET)`. Aquesta línia actua com un rebobinador manual que torna a situar el punter de lectura de l'arxiu al byte inicial exacte (posició 0), permetent reiniciar la descodificació de nou i tancar així un bucle musical infinit.

Exercici 2 - Reproduir un fitxer WAVE a ESP32 des d'una targeta SD externa

Codi

```
#include "Audio.h"
#include "SD.h"
#include "FS.h"

// Connexions digitals utilitzades i adaptades per a l'ESP32-S3
#define SD_CS      10
#define SPI_MOSI   11
#define SPI_MISO   13
#define SPI_SCK    12

#define I2S_DOUT   17
#define I2S_BCLK   18
#define I2S_LRC    16

Audio audio;

void setup(){
  Serial.begin(115200);
  delay(1500);

  Serial.println("\n--- INICIANT REPRODUCCTOR D'ÀUDIO ---");

  pinMode(SD_CS, OUTPUT);
  digitalWrite(SD_CS, HIGH);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, SD_CS);

  Serial.println("Intentant connectar amb la targeta SD...");
  if(!SD.begin(SD_CS, SPI, 4000000)){
    Serial.println("ERROR CRÍTIC: No s'ha pogut inicialitzar la
targeta SD!");
    while(true);
  }
  Serial.println("Targeta SD detectada correctament.");

  audio.setPinout(I2S_BCLK, I2S_LRC, I2S_DOUT);
  audio.setVolume(10);

  Serial.println("Intentant carregar el fitxer d'àudio
prova.wav...");
  // CORRECCIÓ: Afegit el .wav al final
  audio.connecttoFS(SD, "/prova.wav.wav");
}

void loop(){
  audio.loop();
}
```

```
// Funcions de monitorització
void audio_info(const char *info){ Serial.print("INFO ÀUDIO: ");
Serial.println(info); }
void audio_id3data(const char *info){ Serial.print("id3data: ");
Serial.println(info); }
void audio_eof_mp3(const char *info){ Serial.print("Fitxer
finalitzat: "); Serial.> println(info); }
void audio_showstation(const char *info){ Serial.print("station ");
Serial.println> (info); }
void audio_showstreaminfo(const char *info){ Serial.print("streaminfo
"); Serial.println> (info); }
void audio_showstreamtitle(const char *info){
Serial.print("streamtitle "); Serial.> println(info); }
void audio_bitrate(const char *info){ Serial.print("bitrate ");
Serial.println(info); }
void audio_commercial(const char *info){ Serial.print("commercial ");
Serial.println> (info); }
void audio_icyurl(const char *info){ Serial.print("icyurl ");
Serial.println(info); }
void audio_lasthost(const char *info){ Serial.print("lasthost ");
Serial.println> (info); }
void audio_eof_speech(const char *info){ Serial.print("eof_speech ");
Serial.println> (info); }
```

Descripció de la sortida del port serie

Al Monitor Sèrie, la seqüència començarà mostrant el missatge de benvinguda **--- INICIANT REPRODUCCTOR D'ÀUDIO ---** seguit d'Intentant connectar amb la targeta SD.... Si hi ha algun problema de connexió amb el lector de targetes, el programa s'aturarà de manera indefinida llançant un missatge d'"**ERROR CRÍTIC: No s'ha pogut inicialitzar la targeta SD!**". En cas d'èxit, es mostrarà **"Targeta SD detectada correctament"**. i, immediatament després, **"Intentant carregar el fitxer d'àudio prova.wav...."** Durant la reproducció, gràcies a les funcions de monitorització del final del codi, aniran apareixent automàticament diverses línies d'informació tècnica com **"INFO ÀUDIO: (amb detalls del descodificador)"** o bitrate, oferint un registre en temps real del que està sonant.

Explicació del seu funcionament

Aquest programa avança un pas més en els sistemes de so digital implementant un reproductor de fitxers d'àudio (en aquest cas, un fitxer .wav) allotjats en una targeta SD externa. Per aconseguir una reproducció fluida i de gran qualitat, utilitza la potent llibreria **Audio.h** (habitualment coneguda com a ESP32-audioI2S), dissenyada específicament per a microcontroladors potents com l'ESP32-S3. El codi defineix amb precisió dues línies de comunicació totalment independents: el bus SPI per gestionar la memòria de la targeta SD (pins 10, 11, 12 i 13) i el bus I2S per extreure les dades d'àudio pur cap a un xip DAC o amplificador extern (pins 16, 17 i 18).

Dins de la funció **setup()**, es fan les configuracions inicials obligatòries d'arrencada. Primer s'assegura que el pin de selecció de la SD (**SD_CS**) comenci en un estat alt (**HIGH**) per evitar interferències abans d'hora. Després d'iniciar el bus SPI amb els pins personalitzats, el codi executa una condició de seguretat estricta: intenta muntar la targeta SD a una velocitat de 4 MHz; si la targeta no respon, el programa es

bloqueja a propòsit dins d'un bucle buit infinit (`while(true);`) per evitar errors posteriors. Si la targeta funciona bé, es configuren els pins I2S de sortida, s'estableix un volum de 10 i es crida a la funció `audio.connecttoFS(SD, "/prova.wav.wav")` per començar a llegir el fitxer d'àudio especificat.

El cor de l'execució contínua es troba a la funció `loop()`, on només hi ha la instrucció `audio.loop()`. Aquesta funció és altament exigent i s'ha d'executar sense cap mena d'interrupció o retard (`delay`), ja que s'encarrega de rebre els paquets de dades digitals de la SD, descodificar-los en temps real i enviar el corrent de bits constant cap al bus I2S perquè l'altaveu no es quedi mai sense so. Finalment, el codi es complementa amb un conjunt de funcions especials anomenades callbacks (com `audio_info` o `audio_bitrate`). Aquestes funcions es queden escoltant en segon pla i la llibreria les crida automàticament per imprimir pel port sèrie les dades internes del fitxer (com les etiquetes ID3 de títol o artista) i avisar quan la pista s'ha acabat.